Core components

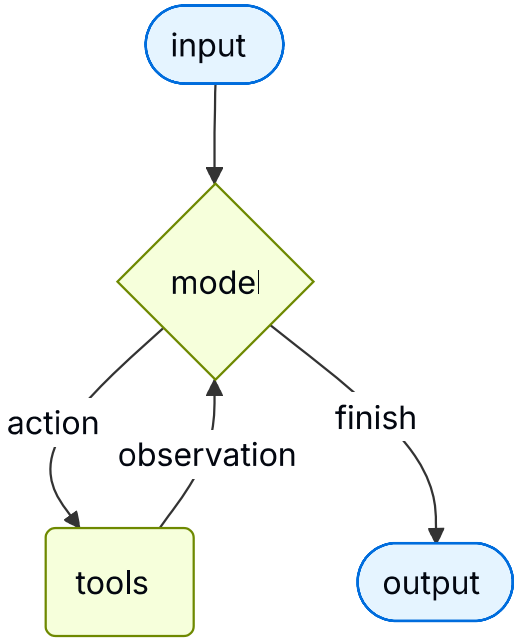
Agents

Copy page

Agents combine language models with tools to create systems that can reason about tasks, decide which tools to use, and iteratively work towards solutions.

`create_agent` provides a production-ready agent implementation.

An LLM Agent runs tools in a loop to achieve a goal. An agent runs until a stop condition is met - i.e., when the model emits a final output or an iteration limit is reached.



`create_agent` builds a **graph**-based agent runtime using LangGraph. A graph consists of nodes (steps) and edges (connections) that define how your agent processes information. The agent moves through this graph, executing nodes like the model node (which calls the model), the tools node (which executes tools), or middleware.

Learn more about the Graph API.

Trace each step of this loop, debug tool calls, and evaluate agent outputs with LangSmith. Follow the tracing quickstart to get set up.

Core components

Model

The model is the reasoning engine of your agent. It can be specified in multiple ways, supporting both static and dynamic model selection.

Static model

Static models are configured once when creating the agent and remain unchanged throughout execution. This is the most common and straightforward approach.

To initialize a static model from a model identifier string:

```python
from langchain.agents import create_agent

agent = create_agent("openai:gpt-5.4", tools=tools)
```

Model identifier strings support automatic inference (e.g., `"gpt-5.4"` will be inferred as `"openai:gpt-5.4"`). Refer to the reference to see a full list of model identifier string mappings.

For more control over the model configuration, initialize a model instance directly using the provider package. In this example, we use `ChatOpenAI`. See Chat models for other available chat model classes.

```
from langchain.agents import create_agent
from langchain_openai import ChatOpenAI

model = ChatOpenAI(
    model="gpt-5.4",
    temperature=0.1,
    max_tokens=1000,
    timeout=30
    # ... (other params)
)
agent = create_agent(model, tools=tools)
```

Model instances give you complete control over configuration. Use them when you need to set specific parameters like `temperature`, `max_tokens`, `timeouts`, `base_url`, and other provider-specific settings. Refer to the reference to see available params and methods on your model.

Dynamic model

Dynamic models are selected at runtime based on the current state and context. This enables sophisticated routing logic and cost optimization.

To use a dynamic model, create middleware using the `@wrap_model_call` decorator that modifies the model in the request:

```python
from langchain_openai import ChatOpenAI
from langchain.agents import create_agent
from langchain.agents.middleware import wrap_model_call, ModelRequest, ModelResponse


basic_model = ChatOpenAI(model="gpt-5.4-mini")
advanced_model = ChatOpenAI(model="gpt-5.4")

@wrap_model_call
def dynamic_model_selection(request: ModelRequest, handler) -> ModelResponse:
    """Choose model based on conversation complexity."""
    message_count = len(request.state["messages"])

    if message_count > 10:
        # Use an advanced model for longer conversations
        model = advanced_model
    else:
        model = basic_model

    return handler(request.override(model=model))

agent = create_agent(
    model=basic_model,  # Default model
    tools=tools,
    middleware=[dynamic_model_selection]
)
```

> ⚠ Pre-bound models (models with `bind_tools` already called) are not supported when using structured output. If you need dynamic model selection with structured output, ensure the models passed to the middleware are not pre-bound.

> 💡 For model configuration details, see Models. For dynamic model selection patterns, see Dynamic model in middleware.

Tools

Tools give agents the ability to take actions. Agents go beyond simple model-only tool binding by facilitating:

- Multiple tool calls in sequence (triggered by a single prompt)
- Parallel tool calls when appropriate
- Dynamic tool selection based on previous results
- Tool retry logic and error handling
- State persistence across tool calls

For more information, see Tools.

Static tools

Static tools are defined when creating the agent and remain unchanged throughout execution. This is the most common and straightforward approach.

To define an agent with static tools, pass a list of the tools to the agent.

Tools can be specified as plain Python functions or coroutines.

The tool decorator can be used to customize tool names, descriptions, argument schemas, and other properties.

```python
from langchain.tools import tool
from langchain.agents import create_agent


@tool
def search(query: str) -> str:
    """Search for information."""
    return f"Results for: {query}"

@tool
def get_weather(location: str) -> str:
    """Get weather information for a location."""
    return f"Weather in {location}: Sunny, 72°F"

agent = create_agent(model, tools=[search, get_weather])
```

If an empty tool list is provided, the agent will consist of a single LLM node without tool-calling capabilities.

Dynamic tools

With dynamic tools, the set of tools available to the agent is modified at runtime rather than defined all upfront. Not every tool is appropriate for every situation. Too many tools may overwhelm the model (overload context) and

increase errors; too few limit capabilities. Dynamic tool selection enables adapting the available toolset based on authentication state, user permissions, feature flags, or conversation stage.

There are two approaches depending on whether tools are known ahead of time:

Filtering pre-registered tools | Runtime tool registration

When all possible tools are known at agent creation time, you can pre-register them and dynamically filter which ones are exposed to the model based on state, permissions, or context.

State | Store | Runtime Context

Enable advanced tools only after certain conversation milestones:

```python
from langchain.agents import create_agent
from langchain.agents.middleware import wrap_model_call, ModelRequest, ModelResponse
from typing import Callable

@wrap_model_call
def state_based_tools(
    request: ModelRequest,
    handler: Callable[[ModelRequest], ModelResponse]
) -> ModelResponse:
    """Filter tools based on conversation State."""
    # Read from State: check if user has authenticated
    state = request.state
    is_authenticated = state.get("authenticated", False)
    message_count = len(state["messages"])

    # Only enable sensitive tools after authentication
    if not is_authenticated:
        tools = [t for t in request.tools if t.name.startswith("public_")]
        request = request.override(tools=tools)
    elif message_count < 5:
        # Limit tools early in conversation
        tools = [t for t in request.tools if t.name != "advanced_search"]
        request = request.override(tools=tools)

    return handler(request)

agent = create_agent(
    model="gpt-5.4",
    tools=[public_search, private_search, advanced_search],
```

```
    middleware=[state_based_tools]
)
```

This approach is best when:

- All possible tools are known at compile/startup time
- You want to filter based on permissions, feature flags, or conversation state
- Tools are static but their availability is dynamic

See Dynamically selecting tools for more examples.

> To learn more about tools, see Tools.

Tool error handling

To customize how tool errors are handled, use the `@wrap_tool_call` decorator to create middleware:

```python
from langchain.agents import create_agent
from langchain.agents.middleware import wrap_tool_call
from langchain.messages import ToolMessage


@wrap_tool_call
def handle_tool_errors(request, handler):
    """Handle tool execution errors with custom messages."""
    try:
        return handler(request)
    except Exception as e:
        # Return a custom error message to the model
        return ToolMessage(
            content=f"Tool error: Please check your input and try again. ({str(e)})",
            tool_call_id=request.tool_call["id"]
        )

agent = create_agent(
    model="gpt-5.4",
    tools=[search, get_weather],
    middleware=[handle_tool_errors]
)
```

The agent will return a `ToolMessage` with the custom error message when a tool fails:

```
[
    ...
    ToolMessage(
        content="Tool error: Please check your input and try again. (division by zero)",
        tool_call_id="..."
    ),
    ...
]
```

Tool use in the ReAct loop

Agents follow the ReAct (“Reasoning + Acting”) pattern, alternating between brief reasoning steps with targeted tool calls and feeding the resulting observations into subsequent decisions until they can deliver a final answer.

Example of ReAct loop

System prompt

You can shape how your agent approaches tasks by providing a prompt. The `system_prompt` parameter can be provided as a string:

```
agent = create_agent(
    model,
    tools,
    system_prompt="You are a helpful assistant. Be concise and accurate."
)
```

When no `system_prompt` is provided, the agent will infer its task from the messages directly.

The `system_prompt` parameter accepts either a `str` or a `SystemMessage`. Using a `SystemMessage` gives you more control over the prompt structure, which is useful for provider-specific features like Anthropic's prompt caching:

```python
from langchain.agents import create_agent
from langchain.messages import SystemMessage, HumanMessage

literary_agent = create_agent(
    model="google_genai:gemini-3.1-pro-preview",
    system_prompt=SystemMessage(
        content=[
            {
                "type": "text",
                "text": "You are an AI assistant tasked with analyzing literary works.",
            },
            {
                "type": "text",
                "text": "<the entire contents of 'Pride and Prejudice'>",
                "cache_control": {"type": "ephemeral"}
            }
        ]
    )
)

result = literary_agent.invoke(
    {"messages": [HumanMessage("Analyze the major themes in 'Pride and Prejudice'.")]}
)
```

The `cache_control` field with `{"type": "ephemeral"}` tells Anthropic to cache that content block, reducing latency and costs for repeated requests that use the same system prompt.

Dynamic system prompt

For more advanced use cases where you need to modify the system prompt based on runtime context or agent state, you can use middleware.

The `@dynamic_prompt` decorator creates middleware that generates system prompts based on the model request:

```python
from typing import TypedDict

from langchain.agents import create_agent
from langchain.agents.middleware import dynamic_prompt, ModelRequest


class Context(TypedDict):
    user_role: str

@dynamic_prompt
def user_role_prompt(request: ModelRequest) -> str:
    """Generate system prompt based on user role."""
    user_role = request.runtime.context.get("user_role", "user")
    base_prompt = "You are a helpful assistant."

    if user_role == "expert":
        return f"{base_prompt} Provide detailed technical responses."
    elif user_role == "beginner":
        return f"{base_prompt} Explain concepts simply and avoid jargon."

    return base_prompt

agent = create_agent(
    model="gpt-5.4",
    tools=[web_search],
    middleware=[user_role_prompt],
    context_schema=Context
)
```

```
# The system prompt will be set dynamically based on context
result = agent.invoke(
    {"messages": [{"role": "user", "content": "Explain machine learning"}]},
    context={"user_role": "expert"}
)
```

For more details on message types and formatting, see Messages. For comprehensive middleware documentation, see Middleware.

Name

Set an optional `name` for the agent. This is used as the node identifier when adding the agent as a subgraph in multi-agent systems:

```
agent = create_agent(
    model,
    tools,
    name="research_assistant"
)
```

Prefer `snake_case` for agent names (e.g., `research_assistant` instead of `Research Assistant`). Some model providers reject names containing spaces or special characters with errors. Using alphanumeric characters, underscores, and hyphens only ensures compatibility across all providers. The same applies to tool names.

Invocation

You can invoke an agent by passing an update to its `State`. All agents include a sequence of messages in their state; to invoke the agent, pass a new message:

```
result = agent.invoke(
    {"messages": [{"role": "user", "content": "What's the weather in San Francisco?"}]}
)
```

For streaming steps and / or tokens from the agent, refer to the streaming guide.

Otherwise, the agent follows the LangGraph Graph API and supports all associated methods, such as `stream` and `invoke`.

Advanced concepts

Structured output

In some situations, you may want the agent to return an output in a specific format. LangChain provides strategies for structured output via the `response_format` parameter.

ToolStrategy

`ToolStrategy` uses artificial tool calling to generate structured output. This works with any model that supports tool calling. `ToolStrategy` should be used when provider-native structured output (via `ProviderStrategy`) is not available or reliable.

```python
from pydantic import BaseModel
from langchain.agents import create_agent
from langchain.agents.structured_output import ToolStrategy


class ContactInfo(BaseModel):
    name: str
    email: str
    phone: str

agent = create_agent(
    model="gpt-5.4-mini",
    tools=[search_tool],
    response_format=ToolStrategy(ContactInfo)
)

result = agent.invoke({
    "messages": [{"role": "user", "content": "Extract contact info from: John Doe, john@example.com, (555) 123-4567"}]
})

result["structured_response"]
# ContactInfo(name='John Doe', email='john@example.com', phone='(555) 123-4567')
```

ProviderStrategy

`ProviderStrategy` uses the model provider's native structured output generation. This is more reliable but only works with providers that support native structured output:

```python
from langchain.agents.structured_output import ProviderStrategy

agent = create_agent(
    model="gpt-5.4",
    response_format=ProviderStrategy(ContactInfo)
)
```

> As of `langchain 1.0`, simply passing a schema (e.g., `response_format=ContactInfo`) will default to `ProviderStrategy` if the model supports native structured output. It will fall back to `ToolStrategy` otherwise.

> To learn about structured output, see Structured output.

Memory

Agents maintain conversation history automatically through the message state. You can also configure the agent to use a custom state schema to remember additional information during the conversation.

Information stored in the state can be thought of as the short-term memory of the agent:

Custom state schemas must extend `AgentState` as a `TypedDict`.

There are two ways to define custom state:

1. Via middleware (preferred)

2. Via `state_schema` on `create_agent`

Defining state via middleware

Use middleware to define custom state when your custom state needs to be accessed by specific middleware hooks and tools attached to said middleware.

```python
from langchain.agents import AgentState
from langchain.agents.middleware import AgentMiddleware
from typing import Any


class CustomState(AgentState):
    user_preferences: dict

class CustomMiddleware(AgentMiddleware):
    state_schema = CustomState
    tools = [tool1, tool2]

    def before_model(self, state: CustomState, runtime) -> dict[str, Any] | None:
        ...

agent = create_agent(
    model,
    tools=tools,
    middleware=[CustomMiddleware()]
)

# The agent can now track additional state beyond messages
result = agent.invoke({
    "messages": [{"role": "user", "content": "I prefer technical explanations"}],
    "user_preferences": {"style": "technical", "verbosity": "detailed"},
})
```

Defining state via `state_schema`

Use the `state_schema` parameter as a shortcut to define custom state that is only used in tools.

```python
from langchain.agents import AgentState


class CustomState(AgentState):
    user_preferences: dict

agent = create_agent(
    model,
    tools=[tool1, tool2],
    state_schema=CustomState
)
# The agent can now track additional state beyond messages
result = agent.invoke({
    "messages": [{"role": "user", "content": "I prefer technical explanations"}],
    "user_preferences": {"style": "technical", "verbosity": "detailed"},
})
```

As of `langchain 1.0`, custom state schemas **must** be `TypedDict` types. Pydantic models and dataclasses are no longer supported. See the v1 migration guide for more details.

Defining custom state via middleware is preferred over defining it via `state_schema` on `create_agent` because it allows you to keep state extensions conceptually scoped to the relevant middleware and tools.

`state_schema` is still supported for backwards compatibility on `create_agent`.

To learn more about memory, see Memory. For information on implementing long-term memory that persists across sessions, see Long-term memory.

Streaming

We've seen how the agent can be called with `invoke` to get a final response. If the agent executes multiple steps, this may take a while. To show intermediate progress, we can stream back messages as they occur.

```
from langchain.messages import AIMessage, HumanMessage

for chunk in agent.stream({
    "messages": [{"role": "user", "content": "Search for AI news and summarize the findings"}]
}, stream_mode="values"):
    # Each chunk contains the full state at that point
    latest_message = chunk["messages"][-1]
    if latest_message.content:
        if isinstance(latest_message, HumanMessage):
            print(f"User: {latest_message.content}")
        elif isinstance(latest_message, AIMessage):
            print(f"Agent: {latest_message.content}")
    elif latest_message.tool_calls:
        print(f"Calling tools: {[tc['name'] for tc in latest_message.tool_calls]}")
```

For more details on streaming, see Streaming.

Middleware

Middleware provides powerful extensibility for customizing agent behavior at different stages of execution. You can use middleware to:

- Process state before the model is called (e.g., message trimming, context injection)
- Modify or validate the model's response (e.g., guardrails, content filtering)
- Handle tool execution errors with custom logic
- Implement dynamic model selection based on state or context
- Add custom logging, monitoring, or analytics

Middleware integrates seamlessly into the agent's execution, allowing you to intercept and modify data flow at key points without changing the core agent logic.

For comprehensive middleware documentation including decorators like `@before_model`, `@after_model`, and `@wrap_tool_call`, see Middleware.

Was this page helpful? Yes No